FAZ 1 · ENVIRONMENT SETUP + YOUR FIRST WEBSITE

İlk Web Sitemiz

Araçlardan yayına: bir web sayfası nasıl hazırlanır, tarayıcıda nasıl açılır?

Sancak takımı
Samet Demir · Çeviri ve mimari slayt
Orkun Yaşar Soner · Teori sunumu
Yusuf Serhat Tümtürk · Canlı kod ve soru-cevap (laboratuvar)

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (MDN Web Docs vb.) kendi orijinal lisanslarına (CC-BY-SA, MIT) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.

Bugün neyi göreceğiz?

1. Ortamı hazırla

Araçları kur
Tarayıcıyı tanı
Kod editörü
Dosyalar ve yollar
Terminal

→

2. Siteyi yap

Planla
HTML: iskelet
CSS: görünüm
JavaScript: davranış

→

3. Yayına al

GitHub Pages
Herkesin açabildiği bir adres

Örnek sitemiz: `serhat-92.github.io` — laboratuvarda bu sitenin kodunu satır satır göreceğiz.

01

ORTAMI HAZIRLA

Kod yazmaya başlamadan önce gerekli araçları kuruyor, çalışma alanımızı düzenliyor ve tarayıcıyı tanıyoruz.

KURULUM • DOSYALAR • TERMİNAL

İşe başlamadan önce: araç çantası

Kod editörü

Kodu yazdığımız program.

MDN önerisi: Visual Studio Code

İki tarayıcı

Kodu test ettiğimiz yer.

En az iki farklı motor: örneğin Firefox ve Chrome

Yerel sunucu

Siteyi kendi bilgisayarımızda sunucudan açar.

VS Code: Live Preview eklentisi

Grafik editörü

Resim kırpma ve küçültme.

İsteğe bağlı, hazır gelen programlar yeter

Benzetme: Aşçı yemeğe başlamadan bıçağını, tezgâhını ve ocağını hazırlar. Biz de koda başlamadan araçlarımızı hazırlarız. Git gibi araçlar ilerleyen adımlarda gelecek.

Kaynak: MDN · Installing basic software

Tarayıcı sayısı değil, motor sayısı önemli

Blink

Chromium tabanlı

Chrome, Edge, Opera, Brave, Vivaldi

Gecko

Mozilla

Firefox

WebKit

Apple

Safari ve iPhone tarayıcıları

✗ Chrome + Edge

Farklı marka, aynı motor (Blink). Bu, iki ayrı test sayılmaz.

✓ Chrome + Firefox

İki farklı motor (Blink + Gecko). Bir motorda çıkan hata diğerinde yakalanır.

Kaynak: MDN · Installing basic software

Neden Word değil de kod editörü?

Word belgesi

Ekranda: Merhaba

Dosyanın içinde: yazı + yazı tipi, boyut, renk, sayfa düzeni gibi görünmeyen biçim bilgileri

Tarayıcı bu dosyayı düzgün okuyamaz.

.html dosyası (düz metin)

Ekranda ve dosyanın içinde aynı şey:

```
<h1>Merhaba</h1>
```

Tarayıcı yazdığımız karakterleri aynen okur.

Sözdizimi renklendirme

Otomatik tamamlama

Prettier + Format On Save

Prettier: kaydettiğimiz anda kodu düzgün girintiye sokar. Kodun ne yaptığını değiştirmez, sadece okunmasını kolaylaştırır.

Klasör yapısı ve dosya yolları

```
first-website/
├── index.html
├── images/
│   └── kali-cubes.jpg
├── styles/
│   └── style.css
└── scripts/
    └── main.js
```

✓ **src="images/kali-cubes.jpg"**

Tarayıcı, index.html'in yanındaki images klasörüne girer ve resmi bulur.

✗ **src="kali-cubes.jpg"**

Tarayıcı resmi index.html'in yanında arar, bulamaz ve yerine alt yazısını gösterir. Bizim yaşadığımız gerçek hata.

Kurallar: küçük harf · boşluk yerine tire (my-file.html) · uzantıları göster · Linux'ta Logo.png ≠ logo.png

Kaynak: MDN · Dealing with files

Terminal bir komutu nasıl çalıştırır?

1. Yazarız

`mkdir images`

→

2. Kabuk okur

Bash, Zsh veya PowerShell komutu çözümler

→

3. Programı bulur

PATH'teki klasörlerde mkdir programını arar

→

4. Çekirdeğe sorar

Program, işletim sisteminden klasör ister (sistem çağrısı)

→

5. Klasör hazır

Diskte images klasörü oluşur

`cd`	klasöre gir	`ls`	listele	`mkdir`	klasör aç
`touch`	boş dosya	`cp`	kopyala	`mv`	taşı / adını değiştir

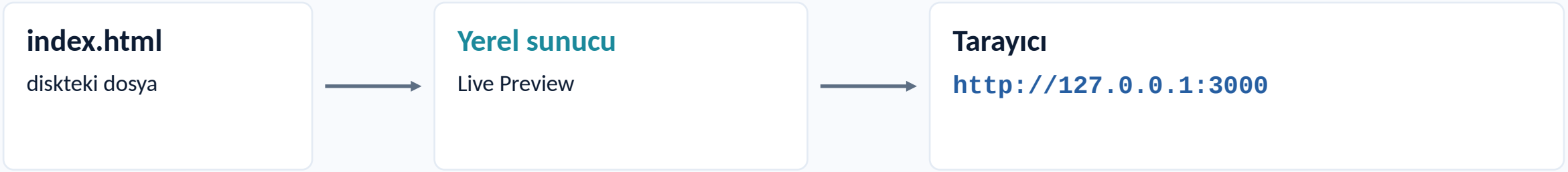
Dosya Gezgini'nde "Yeni Klasör" dediğimizde de sonunda aynı istek yapılır. Fark sadece arayüzdedir.

Kaynak: MDN · Command line

Çift tıklamak ile yerel sunucu arasındaki fark



! Tarayıcı dosyayı doğrudan diskten okur. Güvenlik nedeniyle fetch() ve JavaScript modülleri gibi bazı özellikler engellenebilir.



✓ Dosya, gerçek bir siteden ister gibi HTTP ile istenir. Site, yayındaki haline yakın çalışır.

127.0.0.1 (localhost) = bilgisayarın kendisi. İstek ağ kartından dışarı çıkmaz.

Kaynak: MDN · Installing basic software

Adres çubuğuna yazınca ne olur?

1. DNS
İsim → IP adresi. Rehberden numara bulmak gibi.

→

2. Bağlantı
TCP ile bağlan. https ise TLS ile şifrele.

→

3. HTTP
İstek gönder, yanıt al: 200 tamam, 404 bulunamadı.

→

4. Ek istekler
CSS, JS, resim, font ve favicon.ico için yeni istekler.

→

5. Çizim
DOM + CSSOM → yerleşim → ekrana boyama.

MDN'nin benzetmesi: web bir kütüphanedir

Sunucu = kütüphane	**Web sitesi** = bölüm	**Web sayfası** = kitap	**Arama motoru** = katalog

02

SİTEYİ YAP

Fikrimizi bir web sayfasına dönüştürüyoruz:

HTML ile iskeleti, CSS ile görünümü, JavaScript ile etkileşimi oluşturuyoruz.

PLANLA • KODLA • TEST ET

Kod yazmadan önce: siteyi planla

Konu ve eskiz

Site ne hakkında?

Başlık, resim ve yazının yerini kâğıda kabaca çiz.

Tema rengi

Bir renk seç, kodunu not al:



`#00539f`

Resim

Creative Commons lisanslı olsun.

Boyutuna dikkat: 2,4 MB'lık 4K resim yayında geç yüklendi.

Yazı tipi

Google Fonts'tan seç.

Yüklenemezse yedek yazı tipi devreye girer:

```
"Roboto", sans-serif
```

Neden önemli? Her resim ve her yazı tipi, tarayıcının indirmesi gereken ek bir dosyadır. Tasarım kararı aynı zamanda hız kararıdır.

HTML: sayfanın iskeleti

```
<p class="not">Merhaba</p>
```

Açılış | Özellik (attribute) | İçerik | Kapanış

<head> = zarfın üstü

Ekranda görünmez, tarayıcıya bilgi verir:

charset, viewport, title, CSS ve JS bağlantıları

<body> = mektubun kendisi

Ekranda görünen her şey:

`<h1>, <img alt="...">, <p>, <ul>/<li>, <a href>`

Kaynak: MDN · Creating the content

Tarayıcı HTML'i nasıl okur?

1. Baytlar
Diskten veya ağdan gelen sayılar

→

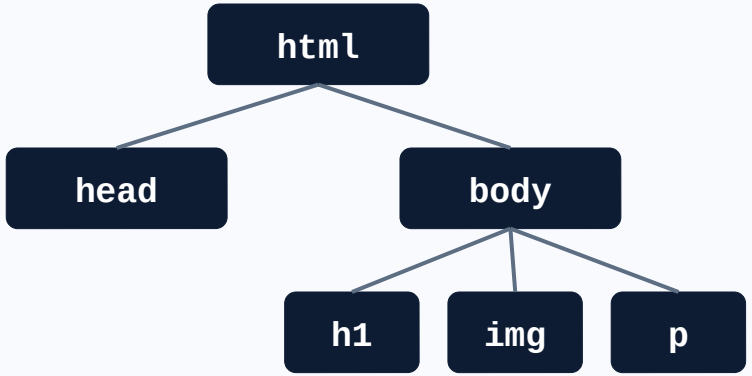
2. Karakterler
charset sayesinde: UTF-8

→

3. Etiketler
<h1>, <p>, <img> tanınır

→

4. DOM ağacı
Bellekteki sayfa haritası



HTML hata vermez, kendince düzeltir

<head> içine yanlışlıkla /* not */ yazdık.

Tarayıcı <head>'i orada kapattı ve notu sayfanın en üstüne bastı.

Doğrusu: `<!-- not -->`

<!doctype html> bir anahtardır

Yoksa tarayıcı eski siteler için uyumluluk moduna (Quirks Mode) geçer.

`document.compatMode`

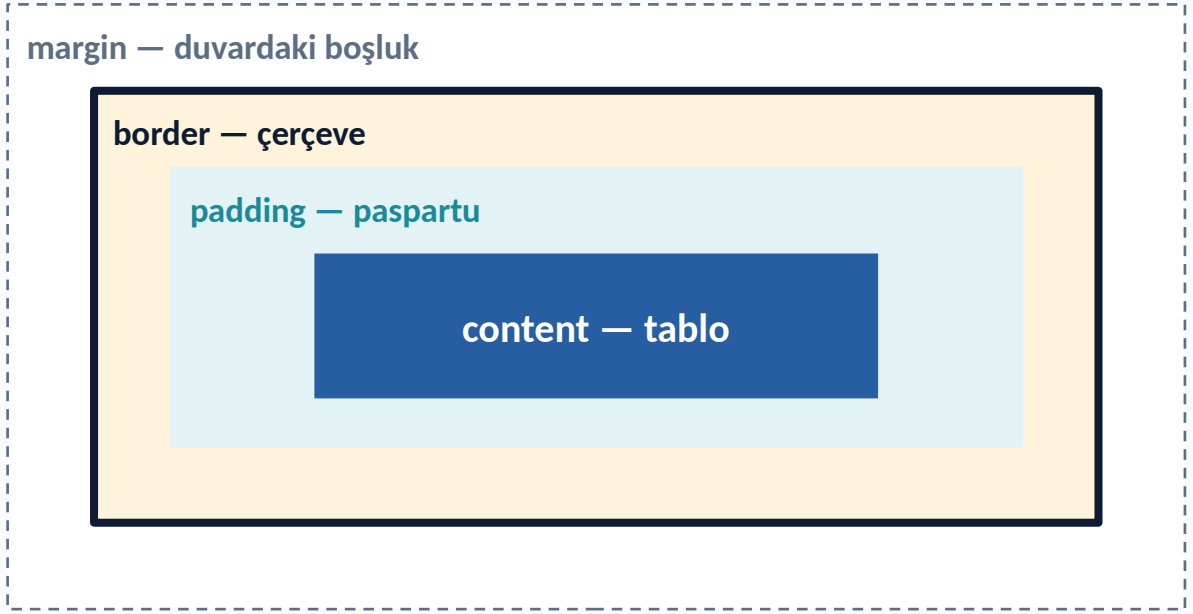
CSS1Compat = standart · BackCompat = Quirks

Kaynak: MDN · Creating the content

CSS: görünüm ve kutu modeli

```
p {
  color: red;
}
```

p = seçici: hangi elementler?
color = özellik: neyi değiştir?
red = değer: neye çevir?



`width: 600px` telefonda taşar. `max-width: 600px` "en fazla 600" der, ekran daralınca kutu da daralır.

JavaScript: sayfaya davranış kazandırmak

1. Tıklama

Kullanıcı resme tıklar

→

2. Dinleyici

`addEventListener("click")`

olayı yakalar

→

3. Fonksiyon

src değerini öbür resimle değiştirir

→

4. DOM güncellenir

Resim değişir, sayfa yenilenmez

Kişisel karşılama

prompt() ile adı sorar.

localStorage ile saklar: sayfa kapansa da hatırlar.

Sonraki ziyarette: "Hoş geldiniz, Serhat"

Bizim iki küçük iyileştirmemiz

`defer`

JS, HTML tamamen okunduktan sonra çalışır.

`return`

İptal'e basılınca pencere tekrar tekrar açılmaz.

Kaynak: MDN · Adding interactivity

03

YAYINA AL

Hazırladığımız siteyi GitHub Pages ile internete taşıyor ve herkesin açabileceği bir bağlantı elde ediyoruz.

GITHUB • HTTPS • PAYLAŞ

Siteyi yayına almak: GitHub Pages

1. Bilgisayardaki klasör

`first-website/`

→

2. GitHub deposu

`kullanıcıadı.github.io`

→

3. GitHub sunucuları

Siteyi kendiliğinden yayınlar

→

4. Herkesin adresi

`https://serhat-92.github.io`

Depo adı kuralı

Depo adı kullanıcıadı.github.io olunca GitHub onu bir web sitesi olarak yayınlar.

HTTPS hazır gelir

Trafik şifrelenir. Araya giren biri içeriği okuyamaz veya değiştiremez.

Yerelde çalışan bozulabilir

Büyük/küçük harf farkı ve büyük dosyalar yayında ortaya çıkar.

Kaynak: MDN · Publishing your website

KAPANIŞ

Şimdi laboratuvara geçiyoruz

HTML sayfanın iskeleti

CSS sayfanın görünümü

JavaScript sayfanın davranışı

Laboratuvar: canlı kod ve soru-cevap

Yusuf Serhat Tümtürk · serhat-92.github.io